

Notes on the Logic of 1D Hydrodynamic Evolution (Numerical)

1 Goal of a Hydrodynamic Code

A hydrodynamic code evolves fluid quantities in time according to conservation laws. In 1D Euler hydrodynamics, the conserved physical quantities are:

- Mass
- Momentum
- Energy

The governing equation is the conservation-law form:

$$\frac{\partial U}{\partial t} + \frac{\partial F(U)}{\partial x} = 0$$

where:

$$U = \begin{pmatrix} \rho \\ \rho v \\ E \end{pmatrix}$$

is the vector of conserved variables, and

$$F(U) = \begin{pmatrix} \rho v \\ \rho v^2 + p \\ v(E + p) \end{pmatrix}$$

is the flux vector.

2 Primitive vs Conserved Variables

Primitive variables:

$$(\rho, v, p)$$

Conserved variables:

$$(\rho, \rho v, E)$$

Why evolve conserved variables?

Because the Euler equations fundamentally conserve mass, momentum, and energy. Numerical finite-volume methods preserve these naturally.

3 Total Energy

The total energy density is:

$$E = \frac{p}{\gamma - 1} + \frac{1}{2}\rho v^2$$

where:

- first term = internal energy
- second term = kinetic energy

4 Finite Volume Philosophy

Space is divided into cells:

$$i-1, \quad i, \quad i+1$$

Each cell stores conserved quantities: U_i . Fluxes are computed at cell interfaces: $F_{i+1/2}$.

The update equation is:

$$U_i^{n+1} = U_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2} - F_{i-1/2})$$

Interpretation:

- incoming flux increases cell content
- outgoing flux decreases cell content

This is the core logic of hydrodynamic evolution.

5 Interface States

At each interface $i + \frac{1}{2}$, there are two neighboring states:

$$U_L = U_i, \quad U_R = U_{i+1}$$

The physical fluxes are:

$$F_L = F(U_L), \quad F_R = F(U_R)$$

The numerical method determines the actual interface flux from these two states.

6 Lax-Friedrichs (Rusanov) Flux

The numerical flux is defined as:

$$F_{i+1/2} = \frac{1}{2}(F_L + F_R) - \frac{1}{2}\alpha(U_R - U_L)$$

This expression has two parts:

(a) Averaged physical flux

$$\frac{1}{2}(F_L + F_R)$$

This term approximates the clean physical flux.

(b) Diffusion term

$$-\frac{1}{2}\alpha(U_R - U_L)$$

This term stabilizes the method. Without it:

- discontinuities create spurious oscillations
- numerical instability develops
- the simulation may explode

The diffusion term smooths sharp jumps between neighboring cells. This diffusion is strictly numerical/artificial, not physical viscosity.

7 Meaning of the Diffusion Term

Suppose neighboring cells differ strongly, meaning $(U_R - U_L)$ is large. Then the diffusion term becomes large and smooths out the discontinuity.

If neighboring cells are similar, the diffusion becomes weak. Hence, the method automatically stabilizes shocks.

The tradeoff:

- More diffusion = more stability but blurrier shocks.
- Less diffusion = sharper solution but possible instability.

8 Signal Speeds

Disturbances propagate through bulk fluid motion and sound waves. The sound speed relative to the fluid is given by:

$$c_s = \sqrt{\frac{\gamma p}{\rho}}$$

If the fluid itself moves with velocity v , then in the lab frame:

- right-moving wave: $v + c_s$
- left-moving wave: $v - c_s$

Therefore, the maximum signal speed is defined as:

$$|v| + c_s$$

This speed is used directly in calculating the numerical flux and enforcing the CFL condition.

9 CFL Condition

The timestep must satisfy:

$$\Delta t \leq C_{\text{CFL}} \frac{\Delta x}{|v| + c_s}$$

Reason: Information must not travel farther than roughly one cell per timestep. Otherwise, causality on the numerical grid breaks down and instability develops.

Typical values:

$$C_{\text{CFL}} \sim 0.3 - 0.8$$

A common choice for beginners is $C_{\text{CFL}} = 0.5$.

10 Sod Shock Tube Problem

The Sod shock tube is a standard benchmark test problem.

Initial condition:

$$(\rho, v, p) = \begin{cases} (1, 0, 1), & x < 0.5 \\ (0.125, 0, 0.1), & x > 0.5 \end{cases}$$

Initially, the left side has high pressure and the right side has low pressure. Removing the separating membrane generates a:

- shock wave
- rarefaction wave
- contact discontinuity

This test checks whether the numerical code correctly captures conservation, wave propagation, shocks, and overall stability. Almost all hydrodynamic codes are first validated on Sod's problem.

11 Initial Conditions Required in the Code

For every grid cell, you must specify:

$$\rho_i, \quad v_i, \quad p_i$$

Also specify the adiabatic index (γ), grid spacing (Δx), and boundary conditions. Then, convert these primitive variables into conserved variables.

12 Overall Numerical Evolution Loop

The complete logic of the code is:

1. Initialize primitive variables
2. Convert to conserved variables
3. Compute timestep from CFL condition
4. Compute physical fluxes
5. Compute interface numerical fluxes
6. Update conserved variables
7. Recover primitive variables
8. Advance time
9. Repeat

In pseudocode:

```
while t < t_final:
    prim_to_cons()
    compute_dt()
    compute_fluxes()
    update_cells()
    cons_to_prim()
    t += dt
```

13 Deep Conceptual Summary

A hydrodynamic code is fundamentally:

- a conservation-law solver
- operating on discrete cells
- using interface fluxes
- while respecting causality through the CFL condition

The entire numerical method is essentially a systematic way of answering: *“How much mass, momentum, and energy cross each interface during one timestep?”*